```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
from google.colab import files

uploaded = files.upload()

df = pd.read_csv("multiple_linear_regression_dataset_500_rec

df.head()
```

Choose Files multiple_lin…records.csv

multiple_linear_regression_dataset_500_records.csv(text/csv) - 11584 bytes, last modified: 7/5/2026 - 100% done

Saving multiple_linear_regression_dataset_500_records.csv to multiple_linear_reg

	Study_Hours	Attendance	Assignments	Previous_Score	Sleep_Hours	Final_Exam
0	2.5	86	80	71	6.4	
1	8.1	92	51	99	6.6	
2	1.1	97	67	43	4.7	
3	9.3	62	67	87	7.1	
4	4.0	91	84	49	7.2	

Next steps: Generate code with df | New interactive sheet

```
print(df.shape)

print(df.info())

print(df.describe())
```

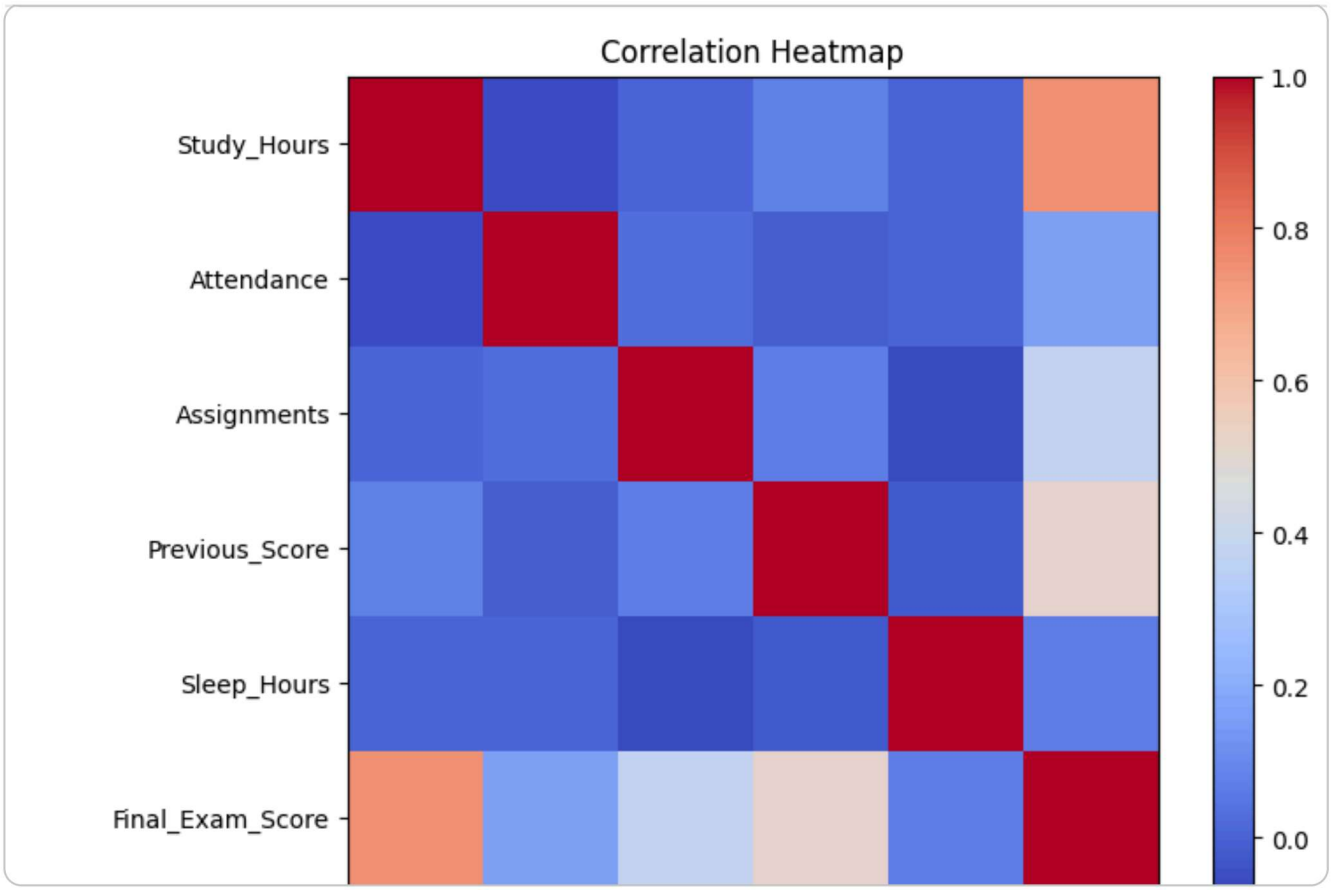
```
(500, 6)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 6 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Study_Hours      500 non-null    float64
 1   Attendance       500 non-null    int64
```

```
 2   Assignments       500 non-null    int64
 3   Previous_Score    500 non-null    int64
 4   Sleep_Hours       500 non-null    float64
 5   Final_Exam_Score  500 non-null    float64
dtypes: float64(3), int64(3)
memory usage: 23.6 KB
None
       Study_Hours  Attendance  Assignments  Previous_Score  Sleep_Hours  \
count   500.000000  500.000000   500.000000      500.000000   500.000000
mean      5.602200   81.460000    69.346000       66.794000     6.500800
std       2.581229   11.717123    17.548966       19.431179     1.464968
min       1.100000   60.000000    40.000000       35.000000     4.000000
25%       3.375000   72.000000    54.000000       49.000000     5.200000
50%       5.650000   83.000000    70.000000       67.000000     6.500000
75%       7.900000   91.000000    84.000000       83.000000     7.800000
max      10.000000  100.000000   100.000000      100.000000     9.000000

       Final_Exam_Score
count        500.000000
mean          74.858320
std           11.478287
min           45.690000
25%           66.117500
50%           74.380000
75%           83.122500
max          102.860000
```

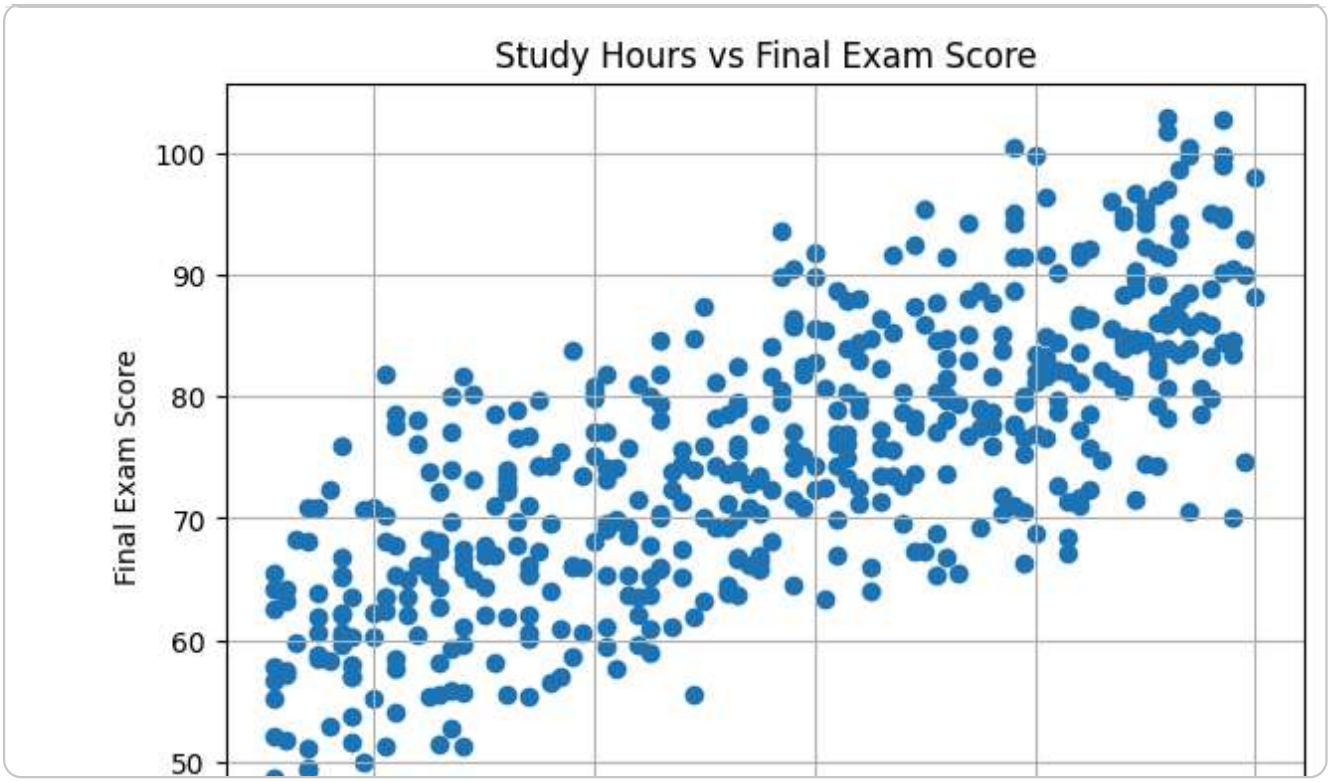
```
print(df.isnull().sum())
```

```
Study_Hours         0
Attendance          0
Assignments         0
Previous_Score      0
Sleep_Hours         0
Final_Exam_Score    0
dtype: int64
```

```
plt.figure(figsize=(8,6))

corr = df.corr()

plt.imshow(corr, cmap='coolwarm', interpolation='nearest')

plt.colorbar()

plt.xticks(range(len(corr.columns)), corr.columns, rotation=

plt.yticks(range(len(corr.columns)), corr.columns)

plt.title("Correlation Heatmap")

plt.show()
```




```
plt.figure(figsize=(7,5))

plt.scatter(df["Study_Hours"], df["Final_Exam_Score"])

plt.xlabel("Study Hours")

plt.ylabel("Final Exam Score")

plt.title("Study Hours vs Final Exam Score")

plt.grid(True)

plt.show()
```




```
X = df.drop("Final_Exam_Score", axis=1)

y = df["Final_Exam_Score"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training Samples:", len(X_train))

print("Testing Samples:", len(X_test))
```

```
Training Samples: 400
Testing Samples: 100
```

```
model = LinearRegression()

model.fit(X_train, y_train)

print("Intercept:", model.intercept_)

coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})
```

```
print(coefficients)
```

```
Intercept: 3.579999648670679
          Feature  Coefficient
0     Study_Hours     3.245463
1      Attendance     0.183736
2     Assignments     0.231710
3  Previous_Score     0.261206
4     Sleep_Hours     0.713147
```

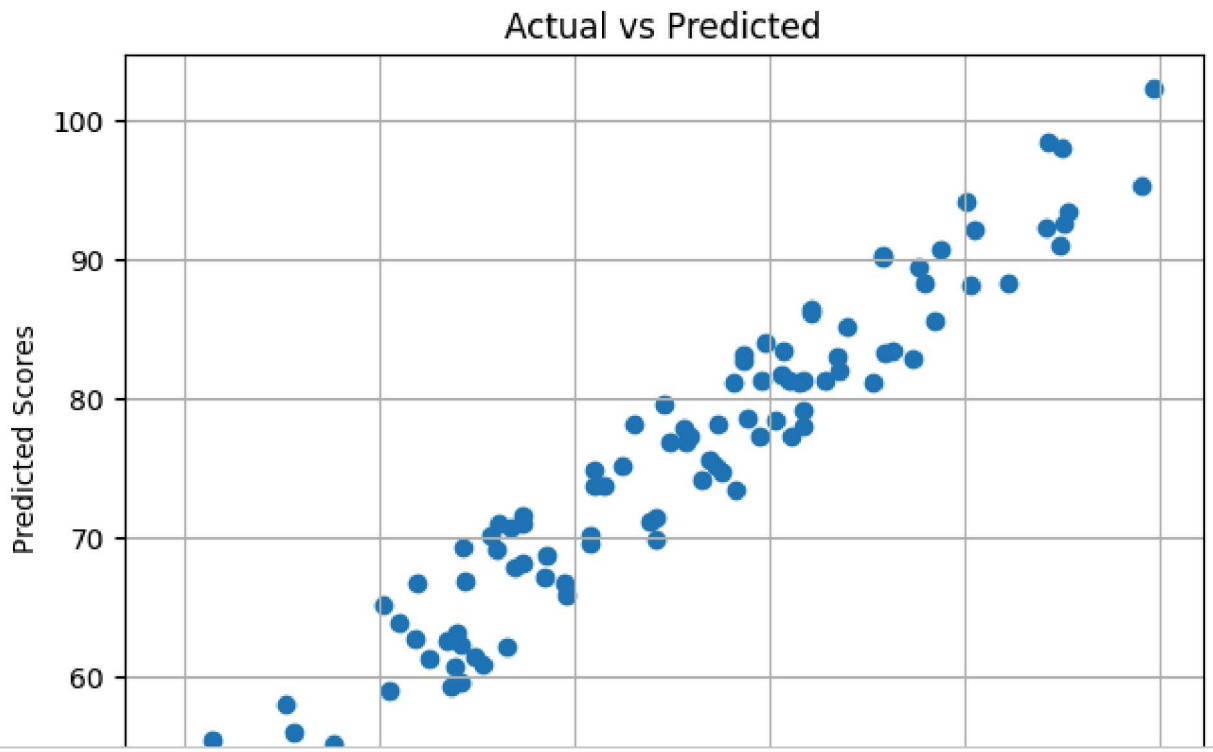
```
y_pred = model.predict(X_test)

prediction = pd.DataFrame({
    "Actual": y_test,
    "Predicted": y_pred
})

prediction.head(10)
```

	Actual	Predicted
361	65.34	60.792871
73	74.17	69.810823
374	86.33	83.427295
155	92.28	88.302118
104	90.16	94.064385
394	95.00	98.041825
377	78.72	82.717234
124	75.96	77.251163
68	66.97	67.798899
450	55.21	57.951043

Next steps: Generate code with `prediction` New interactive sheet

```
plt.figure(figsize=(7,5))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Scores")

plt.ylabel("Predicted Scores")

plt.title("Actual vs Predicted")
```

```
plt.grid(True)

plt.show()
```




```
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("MAE :", mae)

print("MSE :", mse)

print("RMSE:", rmse)

print("R² Score:", r2)
```

```
MAE : 2.7088079821591613
MSE : 9.263519967434194
RMSE: 3.0436031225234004
R² Score: 0.9266707155038799
```

```
residuals = y_test - y_pred

plt.figure(figsize=(7,5))

plt.scatter(y_pred, residuals)
```

```
plt.axhline(y=0, color='red')

plt.xlabel("Predicted Values")

plt.ylabel("Residuals")

plt.title("Residual Plot")

plt.grid(True)

plt.show()
```